

INSTITUTO TECNOLÓGICO DE MORELIA
DEPARTAMENTO INGENIERÍA ELECTRÓNICA



PROFESOR: DR. LUIS ULISES CHÁVEZ CAMPOS
27 DE ABRIL-1 DE MAYO DEL 2026.



LENGUAJES DE INTERFAZ

PRÁCTICA No.6

MASM 8086: OPERACIONES BÁSICAS SOBRE ARCHIVOS DE
DISCO (INT 21H)

1. Objetivo general

Desarrollar en MASM 8086 un programa que realice operaciones básicas sobre archivos de disco en DOS mediante INT 21h: (i) copiar un archivo por bloques (COPY), (ii) contabilizar bytes y líneas durante la copia (WC básico), (iii) aplicar un módulo opcional de procesamiento (conversión a mayúsculas o bitácora), y (iv) mostrar un volcado hexadecimal (hexdump) de los primeros 256 bytes como verificación adicional.

2. Objetivos específicos

Al finalizar la práctica, el estudiante debe ser capaz de:

- Abrir y crear archivos mediante INT 21h usando **handles**.
- Leer y escribir por bloques utilizando un **buffer** fijo.
- Implementar un ciclo de copiado robusto: **leer** → **escribir** → repetir hasta EOF.
- Detectar y manejar errores mediante la bandera de acarreo (**CF**) y el código en **AX**.
- Contar bytes y líneas a partir del flujo de datos leído.
- Transformar el contenido (mayúsculas) o agregar un resumen a un archivo de bitácora.
- Formatear y mostrar bytes en hexadecimal para un hexdump limitado.

3. Conceptos Fundamentales

3.1. Modelo de acceso a archivos en DOS: **handle**

En DOS, al abrir o crear un archivo, el sistema devuelve un **handle** (descriptor) en **AX**. Este handle se utiliza en lecturas/escrituras posteriores. En caso de error, la bandera **CF=1** y **AX** contiene un código de error.

3.2. Operaciones básicas con INT 21h

Las operaciones mínimas requeridas en esta práctica se resumen en la Tabla 1.

Tabla 1. Funciones principales de INT 21h usadas en la práctica.

Función	Registro	Descripción
Abrir archivo	AH=3Dh	Abre un archivo. Entrada: AL modo, DS:DX nombre. Salida: handle en AX.
Crear archivo	AH=3Ch	Crea (o sobrescribe) un archivo. Entrada: CX atributos, DS:DX nombre. Salida: handle en AX.
Leer bloque	AH=3Fh	Lee desde handle. Entrada: BX handle, CX bytes, DS:DX buffer. Salida: bytes leídos en AX (0=EOF).
Escribir bloque	AH=40h	Escribe a handle. Entrada: BX handle, CX bytes, DS:DX buffer.
Cerrar archivo	AH=3Eh	Cierra el handle BX.
Mover puntero (opcional)	AH=42h	Cambia posición (seek): útil para append o lectura parcial.

3.3. Copia por bloques y EOF

La copia eficiente se realiza con un buffer fijo (por ejemplo 256 o 512 bytes). El ciclo termina cuando la lectura devuelve AX=0 (EOF). En cada iteración se escribe exactamente el número de bytes leídos.

3.4. Conteo de líneas

Para conteo de líneas en texto, una aproximación práctica es contar el carácter LF (0Ah) en el buffer. En archivos DOS típicos, una línea termina con CRLF (0Dh 0Ah), por lo que contar 0Ah coincide con el número de líneas.

3.5. Hexdump

Un hexdump muestra el contenido del archivo como bytes hexadecimales, típicamente en grupos de 16 bytes por línea. En esta práctica se limita a los primeros 256 bytes para mantener el alcance de una sesión.

4. Material

4.1. Software

- Emulador: **Emu8086** o DOSBox.
- Ensamblador: MASM (o ensamblador del emulador).
- Editor de texto.

4.2. Archivos requeridos

- Archivo de entrada: `INPUT.TXT` (proporcionado por el alumno).

4.3. Archivos generados por el programa

- Copia: `OUTPUT.TXT`.
- Mayúsculas: `UPPER.TXT`.
- Bitácora: `LOG.TXT`.

5. Requerimientos funcionales

El programa del estudiante debe cumplir lo siguiente:

1. **Parte 1:** abrir `INPUT.TXT` y crear `OUTPUT.TXT`; copiar por bloques usando un buffer.
2. **Parte 2:** durante la copia, calcular:
 - total de bytes copiados,
 - total de líneas (conteo de `0Ah`).
3. **Parte 3:**
 - **Parte 3.1:** generar `UPPER.TXT` convirtiendo letras `a..z` a `A..Z` durante la copia.
 - **Parte 3.2:** agregar un resumen al final de `LOG.TXT` (append) con el total de bytes y líneas.
4. **Parte 4:** mostrar en pantalla un hexdump de los primeros 256 bytes de `INPUT.TXT` (16 bytes por línea, 16 líneas).
5. Manejo mínimo de errores: si falla abrir/crear/leer/escribir, mostrar un mensaje y salir limpiamente cerrando handles abiertos.

6. Restricciones

- El programa debe ser `.model small` (16-bit).
- Debe inicializarse `DS` antes de usar `DS:DX`.
- El copiado debe hacerse por bloques (no byte a byte) con buffer fijo.
- El hexdump está limitado a 256 bytes.
- Los procedimientos deben preservar registros que modifiquen (uso de `PUSH/POP`) cuando aplique.

7. Procedimiento

7.1. Parte A: Preparación y diseño

1. Crear un archivo de prueba `INPUT.TXT` con al menos 5 líneas y caracteres variados.
2. Definir tamaño de buffer (recomendado: 256 bytes) y variables para:
 - bytes totales (`word` o `dword` según preferencia),
 - líneas totales (`word`).
3. Definir una estrategia de mensajes de error y salida limpia.

7.2. Parte B: Implementación (Parte 1 y 2)

1. Implementar:
 - `OpenRead` para `INPUT.TXT`,
 - `CreateWrite` para `OUTPUT.TXT`,
 - `ReadBlock` y `WriteBlock`,
 - `CloseHandle`.
2. Implementar el ciclo `COPY`:
 - leer bloque (n bytes),
 - si $n = 0$ terminar (EOF),
 - sumar n a bytes totales,
 - contar `0Ah` en el buffer para sumar líneas,
 - escribir bloque a `OUTPUT.TXT`.

7.3. Parte C: Implementación (Parte 3)

- **Parte 3.1 (UPPER):** antes de escribir cada bloque, recorrer el buffer y convertir `a..z` a `A..Z`.
- **Parte 3.2 (LOG):** al final, abrir/crear `LOG.TXT`, posicionar al final (`seek`) y escribir una línea con bytes y líneas.

7.4. Parte D: Implementación (Parte 4)

1. Leer los primeros 256 bytes del archivo (reabrir `INPUT.TXT` o usar `seek` al inicio).
2. Imprimir 16 líneas; en cada línea imprimir:
 - `offset` (`0x0000`, `0x0010`, ...),
 - 16 bytes en hexadecimal,
 - (opcional) columna ASCII (caracteres imprimibles o `.`).

7.5. Parte E: Verificación

- Verificar que `OUTPUT.TXT` coincide con `INPUT.TXT`.
- Verificar que bytes/líneas reportados coinciden con el contenido.
- Si se implementó `UPPER.TXT`, verificar conversión correcta de letras.
- Si se implementó `LOG.TXT`, verificar que se agrega una línea y no se sobrescribe.

8. Resultados y Análisis

Adjuntar evidencia de ejecución del programa (captura de pantalla) mostrando al menos: (i) bytes totales copiados, (ii) líneas totales, y (iii) el hexdump (si se realizó). En el análisis, el estudiante debe describir el flujo de copiado por bloques, cómo se detecta EOF, cómo se cuenta LF en el buffer, y cómo se manejan errores mediante CF y el código en AX.

9. Plantilla.

Se proporciona el **Código 1** como plantilla. El estudiante debe completar las rutinas de archivo, el ciclo de copia y los módulos.

Código 1. Plantilla: COPY + WC + (UPPER o LOG) + HEXDUMP.

```

1  ; P6 - Archivos en DOS con INT 21h
2  ; Parte 1: COPY INPUT.TXT -> OUTPUT.TXT (por bloques)
3  ; Parte 2: contar bytes y lineas (LF=0Ah)
4  ; Parte 3: UPPER.TXT y LOG.TXT (append)
5  ; Parte 4: EXTRA: hexdump primeros 256 bytes
6
7  .model small
8  .stack 100h
9
10 .data
11 BUF_SIZE equ 256
12
13 inName db "INPUT.TXT",0
14 outName db "OUTPUT.TXT",0
15 upName db "UPPER.TXT",0
16 logName db "LOG.TXT",0
17
18 buf db BUF_SIZE dup(0)
19
20 hIn dw 0
21 hOut dw 0
22
23 bytesTot dw 0
24 linesTot dw 0
25
26 msgErr db 13,10,"ERROR de archivo.",13,10,'$'
27 msgDone db 13,10,"OK. Copia terminada.",13,10,'$'
28
29 .code
30
31 ; TODO: PrintString (AH=09h)
32 ; TODO: PrintHexWord o PrintDecWord (para reportar totales)
33
34 ; TODO: OpenRead: DS:DX->nombre, devuelve AX=handle (CF=1 error)
35 ; TODO: CreateWrite: DS:DX->nombre, devuelve AX=handle
36 ; TODO: ReadBlock: BX=handle, DS:DX=buf, CX=BUF_SIZE, devuelve AX=bytes leidos
37 ; TODO: WriteBlock: BX=handle, DS:DX=buf, CX=n, devuelve AX=bytes escritos
38 ; TODO: CloseHandle: BX=handle
39
40 ; TODO: CountLF: cuenta 0Ah en buf[0..n-1], devuelve incremento
41 ; TODO (OPCION A): ToUpperBuffer: convierte a..z -> A..Z en buf[0..n-1]
42 ; TODO (EXTRA): HexDump256: imprime 16 lineas x 16 bytes
43
44 main PROC
45     mov ax, @data
46     mov ds, ax
47

```

```
48 ; -----
49 ; Parte 1: abrir INPUT y crear OUTPUT
50 ; -----
51 lea dx, inName
52 ; call OpenRead
53 ; guardar handle en hIn
54
55 lea dx, outName
56 ; call CreateWrite
57 ; guardar handle en hOut
58
59 CopyLoop:
60 ; -----
61 ; Leer bloque
62 ; BX = hIn
63 ; DS:DX = buf
64 ; CX = BUF_SIZE
65 ; call ReadBlock
66 ; AX = n
67 ; if n==0 -> EOF
68 ; -----
69
70 ; -----
71 ; bytesTot += n
72 ; linesTot += CountLF(buf,n)
73 ; -----
74
75 ; -----
76 ; Parte 3 (Parte 3.1): ToUpperBuffer(buf,n)
77 ; -----
78
79 ; -----
80 ; Escribir bloque
81 ; BX = hOut
82 ; DS:DX = buf
83 ; CX = n
84 ; call WriteBlock
85 ; -----
86
87 jmp CopyLoop
88
89 DoneEOF:
90 ; cerrar handles (hIn, hOut)
91
92 ; imprimir bytesTot y linesTot
93
94 ; Parte 4 (extra): HexDump primeros 256 bytes
95 ; - reabrir INPUT y leer 256 bytes
96 ; - llamar HexDump256
97
98 ; salida limpia
99 mov ax, 4C00h
100 int 21h
101 main ENDP
102
```

103 | `end main`

10. Entregables

- Archivo `p6.asm`.
- Archivo generado `OUTPUT.TXT`.
- (Opcional) `UPPER.TXT` o `LOG.TXT`.
- Captura de pantalla con resultados (bytes, líneas y hexdump).

11. Reporte

Las entregas se realizan ocho días después de finalizar la práctica. La calificación del reporte se realiza de acuerdo a los siguientes puntos:

- Claridad y coherencia en la redacción.
- Cumplimiento de los objetivos planteados en la práctica.
- Análisis crítico y reflexivo de los resultados obtenidos.
- Presentación formal y ordenada del documento, de acuerdo al documento guía proporcionado.